Experiment No. 3
Implement a basic grid-world environment as an MDP and applying policy iteration and value iteration algorithms to find optimal policies.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Implementing a basic grid-world environment as an MDP and applying policy iteration and value iteration algorithms to find optimal policies.

Objective: The objective is to create a simplified grid-world environment, representing it as a Markov Decision Process (MDP), and then employ policy iteration and value iteration algorithms to compute optimal policies within this environment, facilitating a fundamental understanding of dynamic programming techniques in reinforcement learning.

Theory:

Markov Decision Process (MDP):

1. An MDP is a mathematical framework used to model decision-making in environments where outcomes are partially random and partially under the control of an agent.
2. It consists of a set of states, a set of actions, transition probabilities, rewards, and a discount factor.
3. In a grid-world environment, each cell represents a state, and the agent can take actions (move up, down, left, right) to transition between states.

Optimal Policies:

4. An optimal policy specifies the best action to take in each state to maximize cumulative rewards over time.
5. Finding optimal policies involves determining the best strategy for the agent to navigate the environment and achieve its goals.

Policy Iteration:

6. Policy iteration is an iterative algorithm for finding the optimal policy in an MDP.
7. It alternates between two steps: policy evaluation and policy improvement.
8. Policy evaluation involves estimating the value function for a given policy, while policy improvement updates the policy based on the current value function.
9. This process continues until the policy converges to an optimal policy.

Algorithm for Policy Iteration:

1. Policy Evaluation:

   - Initialize V(s) arbitrarily for all states

   - Repeat until $\Delta < \varepsilon$ (small positive number):

     - $\Delta \leftarrow 0$

- For each state s:
  - $v \leftarrow V(s)$
  - $V(s) \leftarrow \Sigma(P(s' \mid s, \pi(s)) * [R(s, \pi(s), s') + \gamma * V(s')])$
  - $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

2. Policy Improvement:
   - Policy_stable ← true
   - For each state s:
     - old_action ← $\pi(s)$
     - $\pi(s) \leftarrow \text{argmax}[a]\{\Sigma(P(s' \mid s, a) * [R(s, a, s') + \gamma * V(s')])\}$
     - If old_action $\neq \pi(s)$, then Policy_stable ← false

If Policy_stable, then stop and return optimal policy π^*

Value Iteration:

10. Value iteration is another iterative algorithm used to find optimal policies in MDPs.
11. It iteratively updates the value function for each state until it converges to the optimal value function.
12. The optimal policy can then be derived from the optimal value function by selecting actions that maximize expected returns.

Algorithm of Value Iteration:

Initialize V(s) arbitrarily for all states

Repeat until $\Delta < \varepsilon$ (small positive number):

- $\Delta \leftarrow 0$

For each state s:

- $v \leftarrow V(s)$
- $V(s) \leftarrow \max[a]\{\Sigma(P(s' \mid s, a) * [R(s, a, s') + \gamma * V(s')])\}$
- $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

Return optimal policy π^* such that $\pi^*(s) = \text{argmax}[a]\{\Sigma(P(s' \mid s, a) * [R(s, a, s') + \gamma * V(s')])\}$

In these algorithms:

1. **π** represents the policy, which specifies the action to be taken in each state.
2. **V(s)** represents the value function, which estimates the expected cumulative reward starting from state **s** and following the current policy **π**.
3. **P(s' | s, a)** represents the transition probability from state **s** to state **s'** under action **a**.
4. **R(s, a, s')** represents the reward obtained when transitioning from state **s** to state **s'** under action **a**.
5. **γ** is the discount factor, representing the importance of future rewards compared to immediate rewards.
6. **Δ** is a small positive number used to determine convergence.

These algorithms iteratively update the value function and policy until convergence, where the policy either stabilizes (in policy iteration) or the value function converges (in value iteration). The resulting policy is then considered optimal for the given grid-world environment.

Implementation:

```
import numpy as np


# Step 1: Define Grid World

grid_size = 4

num_states = grid_size * grid_size

actions = [0, 1, 2, 3]  # up, down, left, right

gamma = 0.9


# Step 2: Transition function

def next_state(s, a):

    row, col = divmod(s, grid_size)


    if a == 0 and row > 0: row -= 1
```

```
    elif a == 1 and row < grid_size - 1: row += 1

    elif a == 2 and col > 0: col -= 1

    elif a == 3 and col < grid_size - 1: col += 1



    return row * grid_size + col



# Reward function

def reward(s):

    return 10 if s == num_states - 1 else -1



# ----------------------------

# POLICY ITERATION

# ----------------------------

V = np.zeros(num_states)

policy = np.zeros(num_states, dtype=int)



def policy_evaluation(policy, V):

    theta = 1e-4

    while True:

        delta = 0

        for s in range(num_states):

            v = V[s]
```

```
            a = policy[s]
            s_next = next_state(s, a)
            V[s] = reward(s_next) + gamma * V[s_next]
            delta = max(delta, abs(v - V[s]))
        if delta < theta:
            break
    return V


def policy_improvement(V, policy):
    stable = True
    for s in range(num_states):
        old_action = policy[s]
        q_values = []
        for a in actions:
            s_next = next_state(s, a)
            q_values.append(reward(s_next) + gamma * V[s_next])
        policy[s] = np.argmax(q_values)
        if old_action != policy[s]:
            stable = False
    return policy, stable


# Run Policy Iteration
```

```
while True:
    V = policy_evaluation(policy, V)
    policy, stable = policy_improvement(V, policy)
    if stable:
        break

print("Optimal Policy (Policy Iteration):")
print(policy.reshape(grid_size, grid_size))

# ----------------------------
# VALUE ITERATION
# ----------------------------
V_vi = np.zeros(num_states)

theta = 1e-4
while True:
    delta = 0
    for s in range(num_states):
        v = V_vi[s]
        q_values = []
        for a in actions:
            s_next = next_state(s, a)
```

```
            q_values.append(reward(s_next) + gamma * V_vi[s_next])

        V_vi[s] = max(q_values)

        delta = max(delta, abs(v - V_vi[s]))

    if delta < theta:

        break


# Derive Policy

policy_vi = np.zeros(num_states, dtype=int)

for s in range(num_states):

    q_values = []

    for a in actions:

        s_next = next_state(s, a)

        q_values.append(reward(s_next) + gamma * V_vi[s_next])

    policy_vi[s] = np.argmax(q_values)


print("\nOptimal Policy (Value Iteration):")

print(policy_vi.reshape(grid_size, grid_size))
```

Conclusion:

Difference between Value Iteration and Policy Iteration:

Feature	Policy Iteration	Value Iteration
Approach	Alternates between evaluation & improvement	Directly updates value function
Speed	Fewer iterations but costly per iteration	More iterations but faster per step
Policy	Explicitly updated	Derived after value convergence
Complexity	Higher per iteration	Simpler

Main Steps of Value Iteration:

1. Initialize value function
2. Update value using Bellman optimality equation
3. Repeat until convergence
4. Derive optimal policy from value function